

DOCUMENTACAO_COMPLETA_PTHREADS

Programação Paralela em C com Pthreads: Documentação Técnica Completa

Autor: Especialista em Programação Paralela

Data: 2026

Escopo: Guia técnico, didático e prático de Programação Paralela em C

Índice

1. [Fundamentos](#)
 2. [Processos](#)
 3. [Threads e pthread_create](#)
 4. [pthread_join](#)
 5. [pthread_exit](#)
 6. [Seção Crítica e Mutex](#)
 7. [Semáforos](#)
 8. [Variáveis de Condição](#)
 9. [Barreiras](#)
 10. [Condição de Corrida](#)
 11. [Deadlock](#)
 12. [Escalonamento](#)
 13. [Paralelismo de Dados](#)
 14. [Paralelismo de Tarefas](#)
 15. [Problema: Produtor-Consumidor](#)
 16. [Problema: Leitores-Escritores](#)
 17. [Problema: Jantar dos Filósofos](#)
-

Fundamentos

Execução Sequencial vs. Concorrente vs. Paralela

Definição

- **Execução Sequencial:** Tarefas executadas uma por vez, em ordem estrita.
- **Execução Concorrente:** Múltiplas tarefas em progresso (time-sharing em 1 CPU).

- **Execução Paralela:** Tarefas simultâneas em múltiplos processadores.

Comparação

Modelo	Hardware	Comportamento
Sequencial	1 CPU	T1 → T2 → T3
Concorrente	1 CPU	Intercalação de tarefas
Paralelo	Múltiplos CPUs	Execução simultânea real

Ganho de Desempenho

O paralelismo é vantajoso apenas se o ganho superar o overhead. Tarefas muito pequenas podem ser mais lentas paralelizadas.

Processos

Definição

Um processo é um programa em execução com seu próprio espaço de endereçamento isolado.

Objetivo

Permitir múltiplos programas rodarem independentemente com isolamento total de memória.

Funcionamento

O SO gerencia processos através do **Bloco de Controle de Processo (PCB)**.

Layout de Memória (Formato ELF)

Stack (↓)	Variáveis locais, parâmetros
[Espaço Livre]	
Heap (↑)	malloc/free
.bss / .data	Dados estáticos
.rodata	Constantes

Estados de um Processo

1. **Novo:** Criado, aguardando fila
2. **Pronto:** Aguardando CPU
3. **Execução:** Em processamento
4. **Bloqueado:** Aguardando E/S
5. **Encerrado:** Finalizado

Sintaxe: fork()

```
#include <sys/types.h>
#include <unistd.h>

pid_t fork(void);
```

Parâmetros e Retorno

Retorno	Significado
0	Processo filho
>0	PID do filho (pai recebe)
-1	Erro

Exemplo Completo

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork falhou");
        exit(1);
    } else if (pid == 0) {
        /* CÓDIGO DO FILHO */
        printf("Sou o filho (PID: %d)\n", getpid());
        exit(0);
    } else {
        /* CÓDIGO DO PAI */
    }
```

```
    printf("Sou o pai (PID: %d), aguardando filho...\n", getpid());  
    wait(NULL); /* Sincronização */  
    printf("Filho terminou.\n");  
}  
return 0;  
}
```

Explicação Linha por Linha

1. `fork()` divide o fluxo em dois processos
2. Filho recebe retorno 0, entra no `else if`
3. Pai recebe PID do filho (>0), entra no `else`
4. `wait()` bloqueia pai até filho terminar
5. Cada processo tem seu próprio `getpid()`

Saída Esperada

```
Sou o pai (PID: 1234), aguardando filho...  
Sou o filho (PID: 1235)  
Filho terminou.
```

Problemas Comuns

- **Processo Zumbi:** Filho termina, pai não chama `wait()` → filho fica na tabela de processos
- **Esquecer `wait()`:** Pai continua, deixando filho órfão
- **Confundir `fork()` com `return`:** `fork` retorna EM AMBOS os processos

Erros Frequentes em Provas

- Usar `if (pid == 0)` para identificar pai (incorreto, filho recebe 0)
- Não chamar `wait()` para sincronizar
- Tentar acessar variáveis do pai no filho após `fork` sem entender isolamento de memória

Threads

Definição

Threads (LWP - Light Weight Process) são fluxos de execução independentes que compartilham o **mesmo processo** e espaço de endereçamento.

Objetivo

Permitir paralelismo com baixo custo de criação e facilitar comunicação (compartilham memória).

Funcionamento

Compartilhamento vs. Contexto Privado

Compartilhado	Privado por Thread
Variáveis globais	PC (Program Counter)
Heap	Registradores
Arquivos abertos	Stack
Sinais	TID (Thread ID)

Risco Principal

Acesso simultâneo a dados compartilhados sem sincronização causa **race conditions**.

pthread_create

Definição

Função que cria uma nova thread dentro do processo atual.

Objetivo

Disparar múltiplos fluxos de execução para paralelismo real.

Funcionamento

- **Criação:** Assíncrona (thread chamadora continua imediatamente)
- **Stack:** Cada thread tem sua própria pilha
- **Espaço de Endereçamento:** Compartilhado com todas as threads
- **Escalonamento:** SO decide quando executar cada thread

Sintaxe

```
int pthread_create(  
    pthread_t *thread,  
    const pthread_attr_t *attr,  
    void *(*start_routine)(void *),  
    void *arg  
);
```

Parâmetros

Parâmetro	Significado
*thread	Ponteiro onde armazenar ID da thread
*attr	Atributos (NULL para padrão)
start_routine	Função que a thread executará
arg	Um único argumento (void *)

Valor de Retorno

- **0:** Sucesso
- **Erro:** Código de erro numérico (EAGAIN, ENOMEM, etc.)

Compilação

```
gcc programa.c -o programa -lpthread
```

Exemplo Completo

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int id;
    int valor;
} ThreadData;

void *tarefa(void *arg) {
    ThreadData *data = (ThreadData *)arg;
    printf("Thread %d com valor %d\n", data->id, data->valor);
    free(data);
    return NULL;
}

int main() {
    pthread_t threads[2];

    for (int i = 0; i < 2; i++) {
        ThreadData *td = malloc(sizeof(ThreadData));
        td->id = i;
        td->valor = i * 100;

        if (pthread_create(&threads[i], NULL, tarefa, td) != 0) {
```

```

        perror("Erro ao criar thread");
        return 1;
    }
}

/* Aguardar todas as threads terminarem */
for (int i = 0; i < 2; i++) {
    pthread_join(threads[i], NULL);
}

printf("Todas as threads completadas.\n");
return 0;
}

```

Explicação do Código

1. **Linhas ThreadData:** Struct para passar múltiplos dados
2. **Linha na tarefa():** Recebe `void *`, faz cast para acessar membros
3. **Linha malloc:** Aloca memória NO HEAP (não pode usar variável local)
4. **Linha pthread_create:** Dispara thread com argumento alocado
5. **Linha free:** Thread libera sua própria memória antes de terminar
6. **Linha pthread_join:** Main aguarda todas as threads

Saída Esperada

```

Thread 0 com valor 0
Thread 1 com valor 100
Todas as threads completadas.

```

Problemas Comuns

- **Passar endereço de variável local do loop:** Todas as threads veem o valor final de `i`
- **Esquecer -lpthread:** Erro de ligação
- **Não alocar memória no heap:** Variável local é destruída antes da thread ler

Boas Práticas

1. Sempre verificar retorno de `pthread_create`
2. Para múltiplos argumentos, usar struct
3. Alocar no heap se dados forem usados após `thread_create` retornar
4. Usar `pthread_join` em TODAS as threads

Erros Frequentes em Provas

- Passar `&i` em um loop (race condition)

- Esquecer de compilar com `-lpthread`
 - Não entender que a criação é assíncrona (main pode terminar antes das threads)
-

pthread_join

Definição

Função que bloqueia a thread chamadora até que a thread alvo termine.

Objetivo

Sincronizar threads e coletar resultados.

Funcionamento

- **Bloqueio:** Thread que chama `join` suspende
- **Coleta:** Recebe valor de retorno da thread alvo
- **Limpeza:** Libera recursos da thread encerrada

Sintaxe

```
int pthread_join(pthread_t thread, void **retval);
```

Parâmetros

Parâmetro	Significado
thread	ID da thread a aguardar
**retval	Ponteiro para receber valor de retorno

Valor de Retorno

- **0:** Sucesso
- **Erro:** EINVAL, ESRCH, etc.

Exemplo Completo

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

void *calcular_quadrado(void *arg) {
    int n = *(int *)arg;
```



```

    int *resultado = malloc(sizeof(int));
    *resultado = n * n;
    free(arg); /* Libera argumento recebido */
    return (void *)resultado;
}

int main() {
    pthread_t tid;
    int numero = 5;
    void *ret = NULL;

    /* Alocar para passar para thread */
    int *param = malloc(sizeof(int));
    *param = numero;

    pthread_create(&tid, NULL, calcular_quadrado, param);

    /* pthread_join recebe endereço de void * */
    pthread_join(tid, &ret);

    printf("Resultado: %d\n", *(int *)ret);
    free(ret); /* Obrigatório liberar memória retornada */

    return 0;
}

```

Explicação do Código

1. **Linha calcular_quadrado:** Aloca memória para resultado no heap
2. **Linha return:** Retorna endereço da memória alocada
3. **Linha pthread_join:** Recebe &ret (endereço do ponteiro void *)
4. **Linha depois do join:** ret agora aponta para a memória do resultado
5. **Linha printf:** Faz cast para int* e desreferencia

Saída Esperada

Resultado: 25

Problemas Comuns

- **Dangling Pointer:** Retornar endereço de variável local da thread
- **Memory Leak:** Não liberar memória retornada
- **Zombie Thread:** Não chamar join deixa recursos presos

Boas Práticas

1. **SEMPRE** chamar `pthread_join` ou `pthread_detach`
 2. Se retornar dados, alocar no **HEAP**
 3. Responsabilidade do `join`: liberar memória retornada
-

pthread_exit

Definição

Termina uma thread de forma limpa.

Sintaxe

```
void pthread_exit(void *retval);
```

Objetivo

Permitir que a thread retorne um valor sem encerrar o processo.

Comparação

Função	Efeito
<code>return</code>	Termina função (thread também)
<code>pthread_exit()</code>	Termina apenas a thread
<code>exit()</code>	Termina TODO o processo

Exemplo

```
void *tarefa(void *arg) {  
    int *resultado = malloc(sizeof(int));  
    *resultado = 42;  
    pthread_exit((void *)resultado); /* Termina thread, retorna valor */  
}
```

Nota Especial: pthread_exit() na main

Se `main` chamar `pthread_exit`:

- Main termina
- Processo permanece vivo
- Threads filhas continuam executando
- Processo encerra quando última thread terminar

Seção Crítica e Mutex

Seção Crítica

Definição

Trecho de código que acessa recurso compartilhado que pode ser modificado por múltiplas threads.

Objetivo

Identificar e proteger contra race conditions.

Exemplo de Código Sem Proteção

```
int contador = 0; /* Compartilhado */

void *incrementar(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        contador++; /* SEÇÃO CRÍTICA */
    }
    return NULL;
}
```

Problema: Se 2 threads executarem, esperamos `contador = 2000000`, mas pode ser menor (race condition).

Por que Ocorre?

```
contador++;
/* Traduz-se em: */
/* 1. Carregar contador no registrador (LOAD) */
/* 2. Incrementar registrador (ADD) */
/* 3. Armazenar de volta em memória (STORE) */
/* Se threads se intercalam durante esses 3 passos, há problema! */
```

Mutex (Mutual Exclusion)

Definição

Mecanismo de sincronização que age como um "cadeado" para proteger seções críticas.

Objetivo

Garantir exclusão mútua: apenas uma thread por vez na seção crítica.

Funcionamento

- **Lock:** Obtém cadeado (bloqueia se não disponível)
- **Seção Crítica:** Executa código protegido
- **Unlock:** Libera cadeado

Sintaxe

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

Exemplo Completo

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define N_THREADS 4
#define N_INCREMENTOS 1000000

int contador = 0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *incrementar(void *arg) {
    for (int i = 0; i < N_INCREMENTOS; i++) {
        pthread_mutex_lock(&mutex);
        contador++;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

int main() {
    pthread_t threads[N_THREADS];

    for (int i = 0; i < N_THREADS; i++) {
        pthread_create(&threads[i], NULL, incrementar, NULL);
    }

    for (int i = 0; i < N_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Contador final: %d (esperado: %d)\n",
```

```
        contador, N_THREADS * N_INCREMENTOS);

    pthread_mutex_destroy(&mutex);
    return 0;
}
```

Explicação do Código

1. **Linha mutex global:** Inicializa mutex com estado padrão
2. **Linha lock:** Obtém exclusão (bloqueia se outra thread estiver)
3. **Linha contador++:** Operação atômica agora (só 1 thread por vez)
4. **Linha unlock:** Libera cadeado para próxima thread
5. **Linha destroy:** Libera recursos do mutex ao final

Saída Esperada

```
Contador final: 4000000 (esperado: 4000000)
```

Problemas Comuns

- **Deadlock:** Thread tenta pegar mutex que já possui (sem ser recursivo)
- **Esquecer unlock:** Outras threads ficam bloqueadas para sempre
- **Inconsistência:** Esquecer lock em uma das operações críticas

Boas Práticas

1. **Minimize a seção crítica:**

```
/* BOM: Cálculo fora do lock */
int valor_temp = calculo_pesado();
pthread_mutex_lock(&mutex);
variavel_compartilhada += valor_temp;
pthread_mutex_unlock(&mutex);

/* RUIM: Cálculo dentro do lock */
pthread_mutex_lock(&mutex);
variavel_compartilhada += calculo_pesado();
pthread_mutex_unlock(&mutex);
```

2. **Sempre destruir mutex**
3. **Preferir variáveis locais**

Semáforos

Definição

Primitiva de sincronização baseada em contador inteiro com operações atômicas.

Tipos

Tipo	Descrição
Binário	Valor 0 ou 1 (similar a mutex)
Contador	Valor 0 a N (controla múltiplas instâncias)
Nomeado	Global ao sistema (processos independentes)
Não Nomeado	Na memória do processo (threads)

Operações

- **wait(sem):** Decrementa; se 0, bloqueia
- **post(sem):** Incrementa; acorda thread se houver

Sintaxe

```
#include <semaphore.h>

int sem_init(sem_t *sem, int pshared, unsigned int value);
int sem_wait(sem_t *sem);
int sem_post(sem_t *sem);
int sem_destroy(sem_t *sem);
```

Exemplo: Controle de Recursos (Estacionamento)

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <stdlib.h>

#define N_CARROS 5
#define VAGAS 2

sem_t vagas;

void *carro(void *arg) {
    int id = *(int *)arg;

    printf("Carro %d: tentando entrar...\n", id);
    sem_wait(&vagas); /* Tenta pegar uma vaga */
```

```

    printf("Carro %d: ESTACIONADO.\n", id);
    sleep(2); /* Simula permanência */

    printf("Carro %d: SAINDO...\n", id);
    sem_post(&vagas); /* Libera vaga */

    free(arg);
    return NULL;
}

int main() {
    pthread_t threads[N_CARROS];

    sem_init(&vagas, 0, VAGAS); /* 2 vagas disponíveis */

    for (int i = 0; i < N_CARROS; i++) {
        int *id = malloc(sizeof(int));
        *id = i + 1;
        pthread_create(&threads[i], NULL, carro, id);
    }

    for (int i = 0; i < N_CARROS; i++) {
        pthread_join(threads[i], NULL);
    }

    sem_destroy(&vagas);
    printf("Simulação completada.\n");
    return 0;
}

```

Saída Esperada (Exemplo)

```

Carro 1: tentando entrar...
Carro 1: ESTACIONADO.
Carro 2: tentando entrar...
Carro 2: ESTACIONADO.
Carro 3: tentando entrar...
Carro 1: SAINDO...
Carro 3: ESTACIONADO.
...

```

Diferença entre Mutex e Semáforo

Aspecto	Mutex	Semáforo
Instâncias	1 por vez	N por vez

Aspecto	Mutex	Semáforo
Conceito de Dono	Sim	Não
Uso Típico	Exclusão	Recursos

Variáveis de Condição

Definição

Mecanismo que permite thread suspender até que condição lógica se torne verdadeira.

Objetivo

Evitar "espera ocupada" (busy waiting): em vez de testar repetidamente uma condição, a thread se bloqueia e libera CPU.

Funcionamento

- **Trabalham com Mutex:** Protegem a variável que representa a condição
- **wait:** Libera mutex ATOMICAMENTE e bloqueia
- **signal:** Acorda uma thread
- **broadcast:** Acorda TODAS as threads

Sintaxe

```
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;  
  
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);  
int pthread_cond_signal(pthread_cond_t *cond);  
int pthread_cond_broadcast(pthread_cond_t *cond);
```

Exemplo: Produtor e Consumidor

```
#include <pthread.h>  
#include <stdio.h>  
#include <unistd.h>  
  
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;  
pthread_cond_t c = PTHREAD_COND_INITIALIZER;  
  
int dado_pronto = 0;  
int dado = 0;  
  
void *produtor(void *arg) {
```



```

    sleep(2); /* Simula processamento */

    pthread_mutex_lock(&m);
    dado = 42;
    dado_pronto = 1;
    printf("Produtor: Dado pronto = %d\n", dado);
    pthread_cond_signal(&c); /* Acorda consumidor */
    pthread_mutex_unlock(&m);

    return NULL;
}

void *consumidor(void *arg) {
    pthread_mutex_lock(&m);

    /* SEMPRE use while, não if! */
    while (!dado_pronto) {
        printf("Consumidor: Aguardando...\n");
        pthread_cond_wait(&c, &m); /* Libera m e bloqueia */
    }

    printf("Consumidor: Recebeu %d\n", dado);
    pthread_mutex_unlock(&m);

    return NULL;
}

int main() {
    pthread_t tp, tc;

    pthread_create(&tc, NULL, consumidor, NULL);
    pthread_create(&tp, NULL, produtor, NULL);

    pthread_join(tp, NULL);
    pthread_join(tc, NULL);

    return 0;
}

```

Saída Esperada

```

Consumidor: Aguardando...
Produtor: Dado pronto = 42
Consumidor: Recebeu 42

```

Por que usar `while` não `if` ?

```
/* ERRADO */
if (!dado_pronto)
    pthread_cond_wait(&c, &m);
printf("Consumidor: Recebeu %d\n", dado); /* Pode estar falso! */

/* CORRETO */
while (!dado_pronto)
    pthread_cond_wait(&c, &m);
printf("Consumidor: Recebeu %d\n", dado); /* Garantidamente verdadeiro */
```

Motivo: Despertar espúrio (spurious wakeup) ou múltiplos consumidores podem acordar sem a condição ser verdadeira.

Barreiras

Definição

Ponto de sincronização onde todas as threads de um grupo DEVEM chegar antes que qualquer uma continue.

Objetivo

Coordenar algoritmos que operam em fases (etapas).

Sintaxe

```
int pthread_barrier_init(pthread_barrier_t *barrier,
                        const pthread_barrierattr_t *attr,
                        unsigned count);
int pthread_barrier_wait(pthread_barrier_t *barrier);
int pthread_barrier_destroy(pthread_barrier_t *barrier);
```

Exemplo: Algoritmo de Jacobi

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define N 4
#define NTHREADS 2

double x[N], x_new[N];
pthread_barrier_t barreira;

typedef struct { int id; int start; int end; } thread_data_t;
```

```

void *jacobi_thread(void *arg) {
    thread_data_t *data = (thread_data_t *)arg;

    for (int iteracao = 0; iteracao < 3; iteracao++) {
        /* Fase 1: Cálculo */
        for (int i = data->start; i < data->end; i++) {
            x_new[i] = x[i] + 1.0; /* Cálculo simplificado */
        }

        printf("Thread %d: Calculou fase 1\n", data->id);
        pthread_barrier_wait(&barreira); /* Sincroniza */

        /* Fase 2: Atualização */
        for (int i = data->start; i < data->end; i++) {
            x[i] = x_new[i];
        }

        printf("Thread %d: Atualizou fase 2\n", data->id);
        pthread_barrier_wait(&barreira); /* Sincroniza novamente */
    }

    return NULL;
}

int main() {
    pthread_t threads[NTHREADS];
    thread_data_t data[NTHREADS];

    /* Inicializa array */
    for (int i = 0; i < N; i++) x[i] = 1.0;

    /* Cria barreira para NTHREADS threads */
    pthread_barrier_init(&barreira, NULL, NTHREADS);

    int chunk = N / NTHREADS;
    for (int t = 0; t < NTHREADS; t++) {
        data[t].id = t;
        data[t].start = t * chunk;
        data[t].end = (t == NTHREADS - 1) ? N : (t + 1) * chunk;
        pthread_create(&threads[t], NULL, jacobi_thread, &data[t]);
    }

    for (int t = 0; t < NTHREADS; t++) {
        pthread_join(threads[t], NULL);
    }

    printf("Resultado final: x[0] = %f\n", x[0]);
    pthread_barrier_destroy(&barreira);
}

```

```
    return 0;
}
```

Condição de Corrida (Race Condition)

Definição

Situação onde múltiplas threads acessam mesmo dado compartilhado e pelo menos uma faz escrita, resultando em valor imprevisível.

Objetivo de Compreender

Identificar bugs concorrentes e como evitá-los.

Exemplo Clássico

```
int contador = 0; /* Compartilhado */

void *incrementar(void *arg) {
    for (int i = 0; i < 100000; i++) {
        contador++; /* RACE CONDITION */
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, incrementar, NULL);
    pthread_create(&t2, NULL, incrementar, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Contador = %d (esperado 200000)\n", contador);
    /* Saída típica: 100250, 153472, etc. (NUNCA 200000) */
    return 0;
}
```

Por que Ocorre?

Instrução em nível de máquina:
contador++;

Traduz-se em:

1. LOAD contador → registrador
2. ADD 1 → registrador

3. STORE registrador → contador

Entrelace possível:

T1: LOAD contador (valor 5) → registrador1

T2: LOAD contador (valor 5) → registrador2

T1: ADD 1 → registrador1 (resultado 6)

T2: ADD 1 → registrador2 (resultado 6)

T1: STORE 6 → contador

T2: STORE 6 → contador

Resultado: contador = 6 (deveria ser 7!)

Solução

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *incrementar(void *arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&mutex);
        contador++; /* Agora atômico */
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

Identificação

- Resultado varia a cada execução
- Falta de sincronização em acesso compartilhado
- Pelo menos uma operação é escrita

Deadlock

Definição

Situação onde 2+ threads ficam bloqueadas para sempre, cada uma esperando por recurso que a outra possui.

Condições Necessárias

1. **Exclusão Mútua:** Recurso é exclusivo
2. **Retenção:** Thread retém recurso enquanto aguarda
3. **Não-preempção:** Não há tirado de thread
4. **Espera Circular:** $T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$

Exemplo Clássico

```
pthread_mutex_t m1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m2 = PTHREAD_MUTEX_INITIALIZER;

void *thread1_func(void *arg) {
    pthread_mutex_lock(&m1);
    printf("T1: Tem m1\n");
    sleep(1); /* Garante que T2 vai tentar m2 */

    printf("T1: Tentando m2...\n");
    pthread_mutex_lock(&m2); /* DEADLOCK: T2 tem m2! */
    printf("T1: Tem m2\n");

    pthread_mutex_unlock(&m2);
    pthread_mutex_unlock(&m1);
    return NULL;
}

void *thread2_func(void *arg) {
    pthread_mutex_lock(&m2);
    printf("T2: Tem m2\n");
    sleep(1);

    printf("T2: Tentando m1...\n");
    pthread_mutex_lock(&m1); /* DEADLOCK: T1 tem m1! */
    printf("T2: Tem m1\n");

    pthread_mutex_unlock(&m1);
    pthread_mutex_unlock(&m2);
    return NULL;
}
```

Resultado: Ambas as threads ficam bloqueadas esperando infinitamente.

Solução: Ordem Consistente

```
void *thread1_func(void *arg) {
    pthread_mutex_lock(&m1);
    pthread_mutex_lock(&m2);
    /* ... uso */
    pthread_mutex_unlock(&m2);
    pthread_mutex_unlock(&m1);
}

void *thread2_func(void *arg) {
    pthread_mutex_lock(&m1); /* MESMA ORDEM */
    pthread_mutex_lock(&m2);
```

```
/* ... uso */
pthread_mutex_unlock(&m2);
pthread_mutex_unlock(&m1);
}
```

Detecção

- Programa "trava" sem erro
- Threads ficam em estado "WAITING"
- Nenhuma saída/progresso

Prevenção

1. **Ordem consistente:** Sempre adquirir locks na mesma ordem
 2. **Timeouts:** Usar `pthread_mutex_trylock` com timeout
 3. **Lock-free:** Usar estruturas atômicas
 4. **Timeout em operações I/O**
-

Escalonamento

Definição

Processo pelo qual o SO decide qual thread/processo em estado "Pronto" ganha a CPU.

Algoritmos Comuns

Algoritmo	Descrição	Problema
FCFS	Primeira que chega, primeira a usar	Filas longas se primeira tarefa for pesada
Round Robin (RR)	Cada uma recebe quantum de tempo	Pode gerar muitos context switches
SPN	Tarefa mais curta primeiro	Inanição de tarefas longas
Prioridade	Maior prioridade primeiro	Inanição; mitigado com envelhecimento (aging)

Tipos

Tipo	Característica
Preemptivo	SO pode interromper tarefa (fim quantum, prioridade)
Não-Preemptivo	Tarefa executa até terminar ou bloquear

Quantum e Context Switch

- **Quantum:** Fatia de tempo para cada tarefa (típico: 10-100ms)
- **Context Switch:** Custo de salvar/restaurar estado de thread (overhead)

Implicação para Programação Paralela

- Ordem de execução é **não-determinística**
 - Resultado pode variar entre execuções
 - Sincronização é crítica
-

Paralelismo de Dados

Definição

Múltiplas threads executam **a mesma operação** sobre dados diferentes simultaneamente.

Objetivo

Acelerar processamento de grandes volumes de dados.

Padrão de Execução

Dados: [1, 2, 3, 4, 5, 6, 7, 8]

Threads: 2

T1: [1, 2, 3, 4] → Soma local

T2: [5, 6, 7, 8] → Soma local

Resultado final: Combinar somas

Exemplo Completo: Soma de Vetor

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define N 1000000
#define P 4

float array[N];
float soma_parciais[P];
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
float soma_total = 0;
```



```

typedef struct { int id; int inicio; int fim; } thread_data_t;

void *soma_parcial(void *arg) {
    thread_data_t *data = (thread_data_t *)arg;
    float soma_local = 0;

    /* Cada thread soma sua própria porção SEM LOCK */
    for (int i = data->inicio; i < data->fim; i++) {
        soma_local += array[i];
    }

    /* Seção crítica: atualizar resultado global */
    pthread_mutex_lock(&mutex);
    soma_total += soma_local;
    pthread_mutex_unlock(&mutex);

    free(data);
    return NULL;
}

int main() {
    pthread_t threads[P];

    /* Inicializa array */
    for (int i = 0; i < N; i++) array[i] = 1.0;

    int chunk = N / P;
    for (int i = 0; i < P; i++) {
        thread_data_t *td = malloc(sizeof(thread_data_t));
        td->id = i;
        td->inicio = i * chunk;
        td->fim = (i == P - 1) ? N : (i + 1) * chunk;

        pthread_create(&threads[i], NULL, soma_parcial, td);
    }

    for (int i = 0; i < P; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Soma total = %f (esperado: %f)\n", soma_total, (float)N);
    pthread_mutex_destroy(&mutex);
    return 0;
}

```

Vantagens

- **Escalabilidade:** Adicionar processadores melhora performance
- **Simplicidade:** Cada thread executa mesmo código

- **Comunicação mínima:** Cada thread trabalha independentemente

Limitações

- **Balanceamento de Carga:** Dados podem ter processamento não-uniforme
 - **Overhead:** Se tarefas forem muito pequenas, overhead supera ganho
-

Paralelismo de Tarefas

Definição

Múltiplas threads executam **operações diferentes** (tarefas) simultaneamente.

Objetivo

Explorar paralelismo quando tarefas são naturalmente distintas.

Padrão de Execução

```
T1: Ler arquivo → Processar → Escrever resultado
T2: Ler arquivo → Processar → Escrever resultado
T3: (Diferente) → Validar → Reportar
```

Exemplo: Pipeline de Processamento

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define TAMANHO_FILA 100

typedef struct {
    char dados[256];
    int processado;
} Item;

Item fila[TAMANHO_FILA];
int indice_fila = 0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int fim = 0;

void *produtor(void *arg) {
    for (int i = 0; i < 10; i++) {
```

```

        pthread_mutex_lock(&mutex);

        sprintf(fila[indice_fila].dados, "Item %d", i);
        fila[indice_fila].processado = 0;
        indice_fila++;

        printf("Produtor: Criou item %d\n", i);
        pthread_cond_signal(&cond);
        pthread_mutex_unlock(&mutex);
    }

    pthread_mutex_lock(&mutex);
    fim = 1;
    pthread_cond_broadcast(&cond);
    pthread_mutex_unlock(&mutex);

    return NULL;
}

void *consumidor(void *arg) {
    int meu_id = *(int *)arg;

    while (1) {
        pthread_mutex_lock(&mutex);

        while (indice_fila == 0 && !fim) {
            pthread_cond_wait(&cond, &mutex);
        }

        if (indice_fila == 0 && fim) {
            pthread_mutex_unlock(&mutex);
            break;
        }

        indice_fila--;
        Item item = fila[indice_fila];

        pthread_mutex_unlock(&mutex);

        printf("Consumidor %d: Processando '%s'\n", meu_id, item.dados);
    }

    free(arg);
    return NULL;
}

int main() {
    pthread_t t_prod, t_cons[2];
    int ids[2] = {1, 2};

```

```
pthread_create(&t_prod, NULL, produtor, NULL);
for (int i = 0; i < 2; i++) {
    pthread_create(&t_cons[i], NULL, consumidor, &ids[i]);
}

pthread_join(t_prod, NULL);
for (int i = 0; i < 2; i++) {
    pthread_join(t_cons[i], NULL);
}

return 0;
}
```

Produtor-Consumidor

Definição

Padrão clássico onde **produtores** geram dados e **consumidores** os processam.

Objetivo

Desacoplar produção de consumo, permitindo diferentes velocidades.

Problemas a Resolver

1. **Buffer vazio:** Consumidor não pode ler
2. **Buffer cheio:** Produtor não pode escrever
3. **Sincronização:** Coordenar acesso

Exemplo Completo: Buffer Limitado

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define BUFFER_SIZE 5

typedef struct {
    int items[BUFFER_SIZE];
    int in; /* Posição para inserção */
    int out; /* Posição para extração */
    int count; /* Itens no buffer */
} Buffer;

Buffer buf = {.in = 0, .out = 0, .count = 0};
```

```

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t nao_vazio = PTHREAD_COND_INITIALIZER;
pthread_cond_t nao_cheio = PTHREAD_COND_INITIALIZER;

void *produtor(void *arg) {
    int id = *(int *)arg;

    for (int i = 0; i < 10; i++) {
        int item = id * 100 + i;

        pthread_mutex_lock(&mutex);

        /* Aguarda buffer não estar cheio */
        while (buf.count == BUFFER_SIZE) {
            printf("Produtor %d: Buffer cheio, aguardando...\n", id);
            pthread_cond_wait(&nao_cheio, &mutex);
        }

        /* Insere item */
        buf.items[buf.in] = item;
        buf.in = (buf.in + 1) % BUFFER_SIZE;
        buf.count++;

        printf("Produtor %d: Inseriu %d (count: %d)\n", id, item,
buf.count);

        pthread_cond_signal(&nao_vazio);
        pthread_mutex_unlock(&mutex);

        usleep(100000); /* Simula produção */
    }

    free(arg);
    return NULL;
}

void *consumidor(void *arg) {
    int id = *(int *)arg;

    for (int i = 0; i < 10; i++) {
        pthread_mutex_lock(&mutex);

        /* Aguarda buffer não estar vazio */
        while (buf.count == 0) {
            printf("Consumidor %d: Buffer vazio, aguardando...\n", id);
            pthread_cond_wait(&nao_vazio, &mutex);
        }

        /* Extrai item */
        int item = buf.items[buf.out];

```

```

        buf.out = (buf.out + 1) % BUFFER_SIZE;
        buf.count--;

        printf("Consumidor %d: Consumiu %d (count: %d)\n", id, item,
buf.count);

        pthread_cond_signal(&nao_cheio);
        pthread_mutex_unlock(&mutex);

        usleep(200000); /* Simula consumo */
    }

    free(arg);
    return NULL;
}

int main() {
    pthread_t prod[2], cons[2];
    int ids[4] = {1, 2, 1, 2};

    for (int i = 0; i < 2; i++) {
        int *id = malloc(sizeof(int));
        *id = ids[i];
        pthread_create(&prod[i], NULL, produtor, id);
    }

    for (int i = 0; i < 2; i++) {
        int *id = malloc(sizeof(int));
        *id = ids[2 + i];
        pthread_create(&cons[i], NULL, consumidor, id);
    }

    for (int i = 0; i < 2; i++) {
        pthread_join(prod[i], NULL);
        pthread_join(cons[i], NULL);
    }

    return 0;
}

```

Leitores-Escritores

Definição

Múltiplos leitores podem acessar recurso **simultaneamente**, mas escritores requerem acesso **exclusivo**.

Objetivo

Maximizar concorrência em leitura sem comprometer consistência de escrita.

Restrições

1. Múltiplos leitores → OK
2. Um escritor → Exclui tudo
3. Escritor + leitor → Não permitido

Exemplo Completo

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define MAX_LEITORES 5
#define MAX_ESCRITORES 2

int recurso = 0; /* Dado compartilhado */
int num_leitores = 0;
int escritor_presente = 0;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t pode_ler = PTHREAD_COND_INITIALIZER;
pthread_cond_t pode_escrever = PTHREAD_COND_INITIALIZER;

void *leitor(void *arg) {
    int id = *(int *)arg;

    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&mutex);

        /* Aguarda nenhum escritor */
        while (escritor_presente) {
            printf("Leitor %d: Aguardando escritor sair...\n", id);
            pthread_cond_wait(&pode_ler, &mutex);
        }

        num_leitores++;
        printf("Leitor %d: LENDO (num_leitores: %d, recurso: %d)\n",
            id, num_leitores, recurso);

        pthread_mutex_unlock(&mutex);

        usleep(500000); /* Lê durante 0.5s */
    }
}
```

```

pthread_mutex_lock(&mutex);
num_leitores--;

if (num_leitores == 0) {
    pthread_cond_signal(&pode_escrever);
}

printf("Leitor %d: Terminou leitura\n", id);
pthread_mutex_unlock(&mutex);
}

free(arg);
return NULL;
}

void *escritor(void *arg) {
    int id = *(int *)arg;

    for (int i = 0; i < 2; i++) {
        pthread_mutex_lock(&mutex);

        /* Aguarda nenhum leitor e nenhum escritor */
        while (num_leitores > 0 || escritor_presente) {
            printf("Escritor %d: Aguardando acesso exclusivo...\n", id);
            pthread_cond_wait(&pode_escrever, &mutex);
        }

        escritor_presente = 1;
        printf("Escritor %d: ESCRIVENDO (recurso antigo: %d)\n", id,
recurso);

        pthread_mutex_unlock(&mutex);

        recurso = id * 10 + i; /* Modifica recurso */
        usleep(1000000); /* Escreve durante 1s */

        pthread_mutex_lock(&mutex);
        escritor_presente = 0;

        printf("Escritor %d: Terminou escrita (novo valor: %d)\n", id,
recurso);

        pthread_cond_broadcast(&pode_ler);
        pthread_cond_signal(&pode_escrever);
        pthread_mutex_unlock(&mutex);
    }

    free(arg);
    return NULL;
}

```



```

int main() {
    pthread_t leit[MAX_LEITORES], escrit[MAX_ESCRITORES];

    for (int i = 0; i < MAX_LEITORES; i++) {
        int *id = malloc(sizeof(int));
        *id = i + 1;
        pthread_create(&leit[i], NULL, leitor, id);
    }

    for (int i = 0; i < MAX_ESCRITORES; i++) {
        int *id = malloc(sizeof(int));
        *id = i + 1;
        pthread_create(&escrit[i], NULL, escritor, id);
    }

    for (int i = 0; i < MAX_LEITORES; i++) {
        pthread_join(leit[i], NULL);
    }

    for (int i = 0; i < MAX_ESCRITORES; i++) {
        pthread_join(escrit[i], NULL);
    }

    return 0;
}

```

Saída Esperada (Exemplo)

```

Leitor 1: LENDO (num_leitores: 1, recurso: 0)
Leitor 2: LENDO (num_leitores: 2, recurso: 0)
Leitor 3: LENDO (num_leitores: 3, recurso: 0)
Escritor 1: Aguardando acesso exclusivo...
Leitor 1: Terminou leitura
Leitor 2: Terminou leitura
Leitor 3: Terminou leitura
Escritor 1: ESCREVENDO (recurso antigo: 0)
Escritor 1: Terminou escrita (novo valor: 10)
Leitor 4: LENDO (num_leitores: 1, recurso: 10)
...

```

Jantar dos Filósofos

Definição

Problema clássico de sincronização onde 5 filósofos alternam entre pensar e comer, compartilhando 5 garfos.

Objetivo

Evitar deadlock quando recursos são limitados e threads precisam de múltiplos recursos.

Restrições

- 5 filósofos em torno de uma mesa
- 5 garfos (um entre cada par de filósofos)
- Para comer, precisa de 2 garfos (esquerdo e direito)
- Pode-se pensar com 0 garfos

Problema: Deadlock Trivial

```
void *filosofo(void *arg) {
    int id = *(int *)arg;

    while (1) {
        printf("Filósofo %d: Pensando\n", id);
        sleep(1);

        /* PROBLEMA: Ordem não-coordenada */
        pthread_mutex_lock(&garfo[id]); /* Esquerdo */
        pthread_mutex_lock(&garfo[(id + 1) % 5]); /* Direito */

        printf("Filósofo %d: COMENDO\n", id);
        sleep(1);

        pthread_mutex_unlock(&garfo[id]);
        pthread_mutex_unlock(&garfo[(id + 1) % 5]);
    }
}
```

Deadlock: Todos pegam o garfo esquerdo e ficam esperando o direito!

Solução 1: Ordem Consistente

```
void *filosofo(void *arg) {
    int id = *(int *)arg;

    while (1) {
        printf("Filósofo %d: Pensando\n", id);
        sleep(1);

        int esquerdo = id;
```

```

        int direito = (id + 1) % 5;

        /* Ordena: sempre pega o menor primeiro */
        if (esquerdo > direito) {
            int temp = esquerdo;
            esquerdo = direito;
            direito = temp;
        }

        pthread_mutex_lock(&garfo[esquerdo]);
        pthread_mutex_lock(&garfo[direito]);

        printf("Filósofo %d: COMENDO\n", id);
        sleep(1);

        pthread_mutex_unlock(&garfo[direito]);
        pthread_mutex_unlock(&garfo[esquerdo]);
    }
}

```

Solução 2: Recurso Monitor

```

#define N_FILOSOFOS 5

enum { PENSANDO, COM_FOME, COMENDO } estado[N_FILOSOFOS];

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t pode_comer[N_FILOSOFOS];

void pega_garfos(int id) {
    pthread_mutex_lock(&mutex);

    estado[id] = COM_FOME;
    printf("Filósofo %d: COM FOME\n", id);

    int esquerdo = id;
    int direito = (id + 1) % N_FILOSOFOS;

    /* Aguarda até conseguir ambos os garfos */
    while (estado[esquerdo] == COMENDO ||
           estado[direito] == COMENDO) {
        printf("Filósofo %d: Aguardando garfos...\n", id);
        pthread_cond_wait(&pode_comer[id], &mutex);
    }

    estado[id] = COMENDO;
    printf("Filósofo %d: COMENDO\n", id);

    pthread_mutex_unlock(&mutex);
}

```

```

}

void larga_garfos(int id) {
    pthread_mutex_lock(&mutex);

    estado[id] = PENSANDO;
    printf("Filósofo %d: Voltou a pensar\n", id);

    int esquerdo = id;
    int direito = (id + 1) % N_FILOSOFOS;

    /* Notifica vizinhos que podem ter sido desbloqueados */
    pthread_cond_signal(&pode_comer[esquerdo]);
    pthread_cond_signal(&pode_comer[direito]);

    pthread_mutex_unlock(&mutex);
}

void *filosofo(void *arg) {
    int id = *(int *)arg;

    for (int i = 0; i < 5; i++) {
        pega_garfos(id);
        sleep(1);
        larga_garfos(id);
        sleep(1);
    }

    free(arg);
    return NULL;
}

int main() {
    pthread_t threads[N_FILOSOFOS];

    for (int i = 0; i < N_FILOSOFOS; i++) {
        pthread_cond_init(&pode_comer[i], NULL);
    }

    for (int i = 0; i < N_FILOSOFOS; i++) {
        int *id = malloc(sizeof(int));
        *id = i;
        pthread_create(&threads[i], NULL, filosofo, id);
    }

    for (int i = 0; i < N_FILOSOFOS; i++) {
        pthread_join(threads[i], NULL);
    }
}

```

```
    return 0;
}
```

Saída Esperada (Exemplo)

```
Filósofo 0: Pensando
Filósofo 1: Pensando
Filósofo 2: Pensando
Filósofo 3: COM FOME
Filósofo 3: COMENDO
Filósofo 2: COM FOME
Filósofo 2: Aguardando garfos...
Filósofo 3: Voltou a pensar
Filósofo 2: COMENDO
...
```

Resumo Rápido (Referência)

Conceito	Sintaxe	Objetivo
pthread_create	pthread_create(&tid, NULL, func, arg)	Criar thread
pthread_join	pthread_join(tid, &retval)	Aguardar término
pthread_exit	pthread_exit(retval)	Terminar thread
Mutex Lock	pthread_mutex_lock(&m)	Entrar seção crítica
Mutex Unlock	pthread_mutex_unlock(&m)	Sair seção crítica
Cond Wait	pthread_cond_wait(&c, &m)	Aguardar condição
Cond Signal	pthread_cond_signal(&c)	Acordar 1 thread
Barrier Wait	pthread_barrier_wait(&b)	Sincronizar fases
Sem Wait	sem_wait(&s)	Decrementar semáforo
Sem Post	sem_post(&s)	Incrementar semáforo

Compilação e Execução

```
gcc -pthread programa.c -o programa
./programa
```

Ou:

```
gcc programa.c -o programa -lpthread  
./programa
```

FIM DA DOCUMENTAÇÃO